Cup Manipulator + SEA Cable System
Full State-Space Derivation & C3M Applicability

`script_cup_manipulator_pendulam_tendon_with_spring_sea_pydrake.py`

Isaac-sim robotics notes

April 2026

Contents

1 System Overview **2**

2 Physical Parameters **2**

3 Subsystem 1: Rigid 2R Manipulator Dynamics **2**
- 3.1 Generalized Coordinates . . . 2
- 3.2 Mass Matrix . . . 3
- 3.3 Applied Torques . . . 3

4 Subsystem 2: SEA Motor–Spring–Cable Dynamics **3**
- 4.1 Spring Extension . . . 3
- 4.2 Cable Force (Spring–Damper) . . . 3
- 4.3 Motor Rotor Dynamics (Torque Mode) . . . 4

5 Combined 6D State-Space Model **4**
- 5.1 State and Control Vectors . . . 4
- 5.2 Control-Affine Form . . . 4
  - 5.2.1 Drift term $\boldsymbol{f}(\boldsymbol{x})$ . . . 4
  - 5.2.2 Input matrix $\boldsymbol{B}(\boldsymbol{x})$ . . . 5
- 5.3 Structure of $\boldsymbol{B}$: Row-by-Row . . . 5

6 Underactuation Analysis **5**
- 6.1 Null Space of $\boldsymbol{B}^\top$ . . . 5

7 Current Control: Computed Torque + SEA **6**

8 C3M: Control Contraction Metrics **6**
- 8.1 Background . . . 6
- 8.2 Contraction Condition . . . 6
- 8.3 Dual Conditions (C1 and C2) . . . 7
- 8.4 Why C3M is Well-Suited for This System . . . 7

9 C3M Integration Architecture **7**
- 9.1 Training Phase . . . 7
- 9.2 Deployment in Drake . . . 8
- 9.3 Neural Network Architecture . . . 8

10 Recommended Development Path **9**

1 System Overview

The cup manipulator is a planar 2-DOF serial arm with cable-driven actuation on joint 2 via a Series Elastic Actuator (SEA). The physical topology is:

Joint 1 (shoulder, `link1_base`): Motor shaft → gearbox → rigid link — *no compliance*. Torque τ_1 is applied directly.

Joint 2 (elbow, `link2_link1`): Motor drum → cable → big pulley → **spring** → link 2. The spring (belt elasticity or explicit compliance element) is "in series" between the motor and the joint.

This document derives the complete 6D state-space model in control-affine form $\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{B}(\boldsymbol{x})\,\boldsymbol{u}$ and discusses how C3M (Control Contraction Metrics) can be applied.

2 Physical Parameters

Table 1: Manipulator link parameters (from Onshape URDF export).

Symbol	Description	Value	Unit
L_1	Link 1 length (shoulder → elbow)	334.80	mm
L_2	Link 2 length (elbow → EE)	190.00	mm
m_1	Link 1 mass	5.313	kg
m_2	Link 2 mass	0.520	kg
r_p	Big pulley pitch radius (HTD 5M 60T)	47.75	mm

Table 2: Motor parameters (CubeMars AK60-6 KV80).

Symbol	Description	Value	Unit
N	Gear ratio	6.0	–
J_m	Rotor inertia (motor-side)	3.32×10^{-5}	$\mathrm{kg\,m^2}$
b_m	Motor viscous damping	$0.12/N^2 = 3.33 \times 10^{-3}$	$\mathrm{Nm\,s/rad}$
τ_{peak}	Peak joint torque	9.0	Nm
τ_{cont}	Continuous joint torque	4.5	Nm

Table 3: SEA cable parameters (typical defaults).

Symbol	Description	Default	Unit
k_s	Cable spring stiffness	30–5000	N/m
b_c	Cable dashpot damping	2.0	$\mathrm{N\,s/m}$

3 Subsystem 1: Rigid 2R Manipulator Dynamics

3.1 Generalized Coordinates

Let $\boldsymbol{q} = [q_1, q_2]^\top$ denote the joint angles, where q_1 is the shoulder (link1_base) and q_2 is the elbow (link2_link1).

The standard manipulator equation of motion is:

$$\boldsymbol{M}(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{G}(\boldsymbol{q}) = \boldsymbol{\tau}_{\text{applied}} \tag{1}$$

where $\boldsymbol{M} \in \mathbb{R}^{2\times2}$ is the mass matrix, $\boldsymbol{C} \in \mathbb{R}^{2\times2}$ is the Coriolis matrix, $\boldsymbol{G} \in \mathbb{R}^2$ is the gravity vector, and $\boldsymbol{\tau}_{\text{applied}} \in \mathbb{R}^2$ is the vector of applied joint torques.

3.2 Mass Matrix

For a planar 2R arm with link masses m_1, m_2, lengths L_1, L_2, and centres of mass at distances l_{c1}, l_{c2} along each link:

$$\boldsymbol{M}(\boldsymbol{q}) = \begin{bmatrix} I_1 + I_2 + m_2 L_1^2 + 2m_2 L_1 l_{c2} \cos q_2 & I_2 + m_2 L_1 l_{c2} \cos q_2 \\ I_2 + m_2 L_1 l_{c2} \cos q_2 & I_2 \end{bmatrix} \tag{2}$$

where $I_i = m_i l_{ci}^2$ are the link inertias about their pivot.

3.3 Applied Torques

Crucially, $\boldsymbol{\tau}_{\text{applied}}$ is *not* the controller output.

- **Joint 1** (rigid): $\tau_{1,\text{applied}} = \tau_1$ (direct drive, pass-through)
- **Joint 2** (SEA): $\tau_{2,\text{applied}} = r_p \cdot F_{\text{cable}}$ (spring force, *not* $\tau_{2,\text{des}}$)

The controller commands $[\tau_1, \tau_{2,\text{des}}]$, but $\tau_{2,\text{des}}$ only reaches q_2 through the motor and spring dynamics.

4 Subsystem 2: SEA Motor–Spring–Cable Dynamics

4.1 Spring Extension

The cable wraps on the motor drum (motor-side) and the big pulley (joint-side). In torque mode, the motor-side angle is θ_m (at the rotor, before the gearbox). At the gearbox output, the angle is θ_m/N. The linear cable displacement from the motor is:

$$l_{\text{motor}} = r_p \cdot \frac{\theta_m}{N}$$

and from the joint:

$$l_{\text{joint}} = r_p \cdot q_2$$

The **linear spring extension** is:

$$\boxed{\delta = r_p \left(\frac{\theta_m}{N} - q_2 \right)} \tag{3}$$

4.2 Cable Force (Spring–Damper)

The spring–damper produces a force:

$$F_{\text{raw}} = k_s\, \delta + b_c\, \dot{\delta} \tag{4}$$

where $\dot{\delta} = r_p \left(\dot{\theta}_m/N - \dot{q}_2 \right)$.

Unilateral cable constraint. Cables can only *pull* (tension ≥ 0), never push. The antagonistic cable pair gives:

$$\delta > 0: \quad T_{\text{green}} = \max(F_{\text{raw}}, 0),\ T_{\text{red}} = 0 \qquad \delta < 0: \quad T_{\text{green}} = 0,\ T_{\text{red}} = \max(-F_{\text{raw}}, 0) \tag{5}$$

The net cable force is:

$$F_{\text{cable}} = T_{\text{green}} - T_{\text{red}} \tag{6}$$

and the torque applied to joint 2 is $\tau_{2,\text{applied}} = r_p \cdot F_{\text{cable}}$.

4.3 Motor Rotor Dynamics (Torque Mode)

The motor in MIT torque mode obeys 2nd-order rotor dynamics:

$$\boxed{J_m\,\ddot{\theta}_m = \tau_m - b_m\,\dot{\theta}_m - \frac{r_p\,F_{\text{cable}}}{N}} \tag{7}$$

where the motor-side torque command is:

$$\tau_m = \frac{\tau_{2,\text{des}}}{N}$$

The effective torsional spring stiffness seen by the rotor is $r_p^2 k_s / N^2$, giving a motor-side resonance frequency:

$$\omega_n = \sqrt{\frac{r_p^2\,k_s}{N^2\,J_m}} \tag{8}$$

For AK60-6 with $k_s = 300$ N/m: $\omega_n \approx 24$ rad/s (4 Hz).

5 Combined 6D State-Space Model

5.1 State and Control Vectors

State vector ($n = 6$):

$$\boldsymbol{x} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \\ \theta_m \\ \dot{\theta}_m \end{bmatrix} \in \mathbb{R}^6 \tag{9}$$

Control input ($m = 2$):

$$\boldsymbol{u} = \begin{bmatrix} \tau_1 \\ \tau_{2,\text{des}} \end{bmatrix} \in \mathbb{R}^2 \tag{10}$$

5.2 Control-Affine Form

The system can be written as:

$$\boxed{\dot{\boldsymbol{x}} = \boldsymbol{f}(\boldsymbol{x}) + \boldsymbol{B}(\boldsymbol{x})\,\boldsymbol{u}} \tag{11}$$

5.2.1 Drift term $\boldsymbol{f}(\boldsymbol{x})$

From the manipulator EOM (1) with $\boldsymbol{\tau}_{\text{applied}} = [\tau_1,\ r_p F_{\text{cable}}]^\top$, and eliminating the control terms:

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \\ \left[\boldsymbol{M}^{-1}(\boldsymbol{q})\left(-\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} - \boldsymbol{G}(\boldsymbol{q}) + \begin{pmatrix} 0 \\ r_p\,F_{\text{cable}} \end{pmatrix}\right)\right]_1 \\ \left[\boldsymbol{M}^{-1}(\boldsymbol{q})\left(-\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} - \boldsymbol{G}(\boldsymbol{q}) + \begin{pmatrix} 0 \\ r_p\,F_{\text{cable}} \end{pmatrix}\right)\right]_2 \\ \dot{\theta}_m \\ \dfrac{1}{J_m}\left(-b_m\,\dot{\theta}_m - \dfrac{r_p\,F_{\text{cable}}}{N}\right) \end{bmatrix} \tag{12}$$

Note that $F_{\text{cable}} = F_{\text{cable}}(\boldsymbol{x})$ depends on the state through (3)–(6). The spring force *couples* the motor state $(\theta_m, \dot{\theta}_m)$ to the joint state $(q_2, \dot{q}_2)$ — this coupling lives entirely in $\boldsymbol{f}$, *not* in $\boldsymbol{B}$.

5.2.2 Input matrix $\boldsymbol{B}(\boldsymbol{x})$

Writing $\boldsymbol{M}^{-1}(\boldsymbol{q}) = \begin{bmatrix} M_{11}^{-1} & M_{12}^{-1} \\ M_{21}^{-1} & M_{22}^{-1} \end{bmatrix}$:

$$\boldsymbol{B}(\boldsymbol{x}) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ M_{11}^{-1}(\boldsymbol{q}) & 0 \\ M_{21}^{-1}(\boldsymbol{q}) & 0 \\ 0 & 0 \\ 0 & \dfrac{1}{J_m N} \end{bmatrix} \in \mathbb{R}^{6\times 2} \tag{13}$$

Key observation: $\tau_{2,\text{des}}$ appears *only* in row 6 ($\ddot{\theta}_m$), *not* in row 4 ($\ddot{q}_2$). The commanded torque reaches joint 2 *only through the spring dynamics* in $\boldsymbol{f}(\boldsymbol{x})$. This is the fundamental source of torque lag when k_s is small.

5.3 Structure of $\boldsymbol{B}$: Row-by-Row

Table 4: Input matrix $\boldsymbol{B}(\boldsymbol{x})$ structure.

Row	State derivative	τ_1 column	$\tau_{2,\text{des}}$ column
0	$\dot{q}_1$	0	0
1	$\dot{q}_2$	0	0
2	$\ddot{q}_1$	$M_{11}^{-1}(\boldsymbol{q})$	0
3	$\ddot{q}_2$	$M_{21}^{-1}(\boldsymbol{q})$	**0**
4	$\dot{\theta}_m$	0	0
5	$\ddot{\theta}_m$	0	$1/(J_m N)$

The input matrix has $\text{rank}(\boldsymbol{B}) = 2$ everywhere (since $M_{11}^{-1} > 0$ and $J_m > 0$), but $\tau_{2,\text{des}}$ acts only on the motor rotor.

6 Underactuation Analysis

Table 5: Actuation comparison: rigid vs. SEA manipulator.

Property	Rigid 2R	SEA Cable (Torque mode)
States (n)	4	**6**
Controls (m)	2	2
Unactuated directions ($n-m$)	2	**4**
Fully actuated?	Yes	**No**

The rigid manipulator is fully actuated: both τ_1 and τ_2 act directly on $\ddot{q}_1$ and $\ddot{q}_2$ through the invertible mass matrix, and feedback linearisation (computed torque) is exact.

With the SEA, $\tau_{2,\text{des}}$ only enters $\ddot{\theta}_m$. Joint 2 is driven *indirectly* through the spring coupling in $\boldsymbol{f}$. The system is **underactuated**: we cannot independently command $\ddot{q}_2$.

6.1 Null Space of $\boldsymbol{B}^\top$

The null space $\boldsymbol{B}^\perp$ satisfies $\boldsymbol{B}^\top \boldsymbol{B}^\perp = \boldsymbol{0}$. Since $\boldsymbol{B}$ has rank 2 in $\mathbb{R}^6$, the null space has dimension $n - m = 4$. A basis is:

$$\boldsymbol{B}^{\perp}(\boldsymbol{x}) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & -M_{21}^{-1}/M_{11}^{-1} & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \in \mathbb{R}^{6\times 4} \tag{14}$$

The columns correspond to:

1. $\boldsymbol{e}_1$: the q_1 kinematic identity ($\dot{q}_1 = \dot{q}_1$)
2. Modified $\boldsymbol{e}_2$: the q_2 direction, with row 4 adjusted so the τ_1 coupling through M_{21}^{-1} is cancelled
3. $\boldsymbol{e}_4$: the $\ddot{q}_2$ direction decoupled from τ_1
4. $\boldsymbol{e}_5$: the θ_m kinematic identity

7 Current Control: Computed Torque + SEA

The existing controller in `controller/controller.py` uses a **two-block architecture**:

$$\text{Trajectory} \xrightarrow{\boldsymbol{p}_{\text{ref}}(t)} \text{CT Controller} \xrightarrow{[\tau_1, \tau_{2,\text{des}}]} \text{SEA Actuator} \xrightarrow{[\tau_1, r_p F_{\text{cable}}]} \text{MultibodyPlant}$$

The CT controller computes:

$$\boldsymbol{q}_{\text{des}} = \text{IK}(\boldsymbol{p}_{\text{ref}}) \tag{15}$$
$$\boldsymbol{a}_{\text{des}} = \ddot{\boldsymbol{q}}_{\text{ref}} + K_p\,(\boldsymbol{q}_{\text{des}} - \boldsymbol{q}) + K_d\,(\dot{\boldsymbol{q}}_{\text{ref}} - \dot{\boldsymbol{q}}) \tag{16}$$
$$\boldsymbol{\tau}_{\text{des}} = \boldsymbol{M}(\boldsymbol{q})\,\boldsymbol{a}_{\text{des}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{G}(\boldsymbol{q}) \tag{17}$$

Why CT fails with soft springs: The CT law (17) assumes $\boldsymbol{\tau}_{\text{applied}} = \boldsymbol{\tau}_{\text{des}}$ (instant torque authority). With the SEA, $\tau_{2,\text{applied}} = r_p\,F_{\text{cable}} \neq \tau_{2,\text{des}}$, and the mismatch grows as k_s decreases. The CT controller is *model-blind* to the spring dynamics — it treats the SEA as if it were rigid.

8 C3M: Control Contraction Metrics

8.1 Background

C3M (Sun et al., 2020) jointly learns two neural networks:

1. $\boldsymbol{W}(\boldsymbol{x})$: a state-dependent **dual contraction metric** (positive definite matrix)
2. $\boldsymbol{u}(\boldsymbol{x}, \boldsymbol{x}_e, \boldsymbol{u}_{\text{ref}})$: a **tracking controller** that guarantees exponential convergence at rate λ

where $\boldsymbol{x}_e = \boldsymbol{x} - \boldsymbol{x}_{\text{ref}}$ is the tracking error and $\boldsymbol{u}_{\text{ref}}$ is the feedforward control.

8.2 Contraction Condition

With $\boldsymbol{M} = \boldsymbol{W}^{-1}$ (the contraction metric), the training loss enforces:

$$\dot{\boldsymbol{M}} + (\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K})^{\top}\boldsymbol{M} + \boldsymbol{M}\,(\boldsymbol{A} + \boldsymbol{B}\boldsymbol{K}) + 2\lambda\,\boldsymbol{M} \preceq 0 \tag{18}$$

where $\boldsymbol{A} = \partial \boldsymbol{f}/\partial \boldsymbol{x} + \sum_i u_i\,\partial \boldsymbol{B}_i/\partial \boldsymbol{x}$ and $\boldsymbol{K} = \partial \boldsymbol{u}/\partial \boldsymbol{x}$. This ensures the distance between any two trajectories (measured by $\boldsymbol{M}$) contracts exponentially at rate λ.

8.3 Dual Conditions (C1 and C2)

C3M decomposes the contraction condition into two parts projected through $\boldsymbol{B}^\perp$:

C1 — Unactuated contraction.

$$(\boldsymbol{B}^\perp)^\top \left[-\dot{\boldsymbol{W}}\boldsymbol{f} + \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\boldsymbol{W} + \boldsymbol{W}\left(\frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}\right)^\top + 2\lambda \boldsymbol{W} \right] \boldsymbol{B}^\perp \preceq 0 \tag{19}$$

This must hold in the **4-dimensional null space** of $\boldsymbol{B}^\top$. For the SEA system, this subspace includes the q_2 and θ_m directions — so C3M must find a metric that certifies the *spring dynamics naturally contract* even though we cannot directly command $\ddot{q}_2$.

C2 — Actuated orthogonality.

$$(\boldsymbol{B}^\perp)^\top \left[\dot{\boldsymbol{W}}\boldsymbol{B}_j - \frac{\partial \boldsymbol{B}_j}{\partial \boldsymbol{x}}\boldsymbol{W} - \boldsymbol{W}\left(\frac{\partial \boldsymbol{B}_j}{\partial \boldsymbol{x}}\right)^\top \right] \boldsymbol{B}^\perp = 0 \quad \forall\, j = 1, \ldots, m \tag{20}$$

This ensures the control directions are metrically orthogonal to the unactuated subspace.

8.4 Why C3M is Well-Suited for This System

Table 6: Comparison of control approaches for the cup manipulator + SEA.

Property	CT	CT + RL residual	LQR	**C3M**
Accounts for spring dynamics	No	Learned	Linear only	**Yes**
Guaranteed convergence rate	No	No	Local only	**Yes** (λ)
Handles underactuation	No	Partially	Partially	**Yes**
Robust to model uncertainty	No	Partially	No	**Certified**
Need training	No	Yes	No	Yes

Key advantages of C3M for this specific system:

1. **Spring dynamics awareness.** The C1 condition (19) explicitly verifies that the spring coupling from θ_m to q_2 is contracting. If k_s is too small, C1 becomes infeasible for a given λ — this gives a *principled lower bound on spring stiffness*.

2. **Anticipatory control.** Unlike CT (which ignores the spring lag), C3M learns a controller that *anticipates* the motor-spring dynamics. The controller $\boldsymbol{u}(\boldsymbol{x}, \boldsymbol{x}_e, \boldsymbol{u}_{\text{ref}})$ sees the full state including $(\theta_m, \dot{\theta}_m)$ and can pre-compensate for the spring lag.

3. **Convergence rate certificate.** The parameter $\lambda > 0$ is a guaranteed convergence rate. For tracking tasks, this translates to a bound on the steady-state tracking error: fast trajectories with low λ will have larger error, providing a principled speed–accuracy tradeoff.

4. **Unilateral cable handling.** The $\max(F, 0)$ nonlinearity can be smoothed via softplus $\tilde{F} = \frac{1}{\beta}\ln(1 + e^{\beta F})$ during training to maintain differentiability for PyTorch autograd, while the deployment uses exact max.

9 C3M Integration Architecture

9.1 Training Phase

Three files must be created in the C3M framework (`contraction-theory/C3M/`):

1. `systems/system_CUPMANIP_SEA.py` — 6D $\boldsymbol{f}(\boldsymbol{x})$, $\boldsymbol{B}(\boldsymbol{x})$, $\boldsymbol{B}^{\perp}(\boldsymbol{x})$ in PyTorch
2. `configs/config_CUPMANIP_SEA.py` — state/control bounds, sampling ranges
3. `models/model_CUPMANIP_SEA.py` — neural network architecture for $\boldsymbol{W}$ and $\boldsymbol{u}$

The $\boldsymbol{f}(\boldsymbol{x})$ implementation requires symbolic 2R manipulator EOM in PyTorch (not Drake), since autograd must compute $\partial \boldsymbol{f}/\partial \boldsymbol{x}$.

Training command:

```
cd contraction-theory/C3M
python main.py --task CUPMANIP_SEA --lambda 1.5 \
    --log checkpoints/cupmanip_sea/lambda_1.5 --epochs 15
```

9.2 Deployment in Drake

The trained C3M controller replaces the CT controller as a Drake `LeafSystem`:

$$\text{Trajectory} \xrightarrow{\boldsymbol{x}_{\text{ref}}(t)} \texttt{C3MController} \xrightarrow{[\tau_1, \tau_{2,\text{des}}]} \underbrace{\texttt{SEACableActuator}}_{\text{physical spring model}} \xrightarrow{[\tau_1, r_p F]} \texttt{Plant}$$

The SEA actuator block *remains unchanged* — it models physical reality. The C3M controller produces torque commands that account for the spring dynamics, rather than blindly inverting the manipulator dynamics and hoping the spring keeps up.

The reference state includes the motor:

$$\boldsymbol{x}_{\text{ref}}(t) = \begin{bmatrix} q_{1,\text{ref}}(t) \\ q_{2,\text{ref}}(t) \\ \dot{q}_{1,\text{ref}}(t) \\ \dot{q}_{2,\text{ref}}(t) \\ N \cdot q_{2,\text{ref}}(t) \\ N \cdot \dot{q}_{2,\text{ref}}(t) \end{bmatrix} \tag{21}$$

where $\theta_{m,\text{ref}} = N \cdot q_{2,\text{ref}}$ corresponds to the spring-at-rest equilibrium ($\delta = 0$).

9.3 Neural Network Architecture

The controller network in C3M takes an `effective_dim` slice of the state as input. For the cart-pendulum (4D state, 1D control), dimensions 1–3 ($\dot{p}, \theta, \dot{\theta}$) are used (cart position p is ignored since the dynamics are translation-invariant).

For the cup manipulator SEA (6D state, 2D control), **all 6 states matter** because the dynamics depend on all angles and the motor state:

$$\texttt{effective\_dim\_start} = 0, \quad \texttt{effective\_dim\_end} = 6$$

The network dimensions scale accordingly:

- $\boldsymbol{W}$: $\texttt{Linear}(6, 128) \to \tanh \to \texttt{Linear}(128, 36)$ $\quad (6 \times 6 = 36 \text{ entries})$
- $\boldsymbol{u}_{w1}$: $\texttt{Linear}(12, 128) \to \tanh \to \texttt{Linear}(128, 6c)$ $\quad (\text{input: } [\boldsymbol{x}_{\text{eff}}, \boldsymbol{x}_{e\text{eff}}],\ c = 3n = 18)$
- $\boldsymbol{u}_{w2}$: $\texttt{Linear}(12, 128) \to \tanh \to \texttt{Linear}(128, 2c)$ $\quad (\text{output: } 2 \times c \text{ matrix})$

10 Recommended Development Path

1. **Phase 1: Rigid 4D system.** Start with the rigid manipulator ($n = 4, m = 2$, fully actuated). This should train easily and validate the PyTorch dynamics against Drake. $\boldsymbol{B}^{\perp}$ is only 4×2 (two kinematic identities).

2. **Phase 2: 6D SEA system.** Add the motor states. The 4D null space and spring nonlinearity make training harder. Use softplus approximation for $\max(F, 0)$. Start with stiff springs ($k_s = 500$) and gradually reduce.

3. **Phase 3: Feasibility boundary.** Sweep λ vs. k_s to find the region where C1 is satisfiable. This gives a principled minimum-stiffness curve for a given convergence rate — useful for hardware design.

4. **Phase 4: Drake deployment.** Package the trained C3M controller as a Drake `LeafSystem`, compare against CT + SEA and CT + RL residual on the same rectangular tracking trajectory.